AdVard — Comprehensive Technical Functionality Report

Detailed Architecture, Mechanism of Action, and ADB Implementation Specs for all Desktop Android Controller Capabilities

1. Executive Overview & System Architecture

AdVard is a cross-platform desktop control suite designed for seamless remote administration of Android smartphones over a physical USB connection. Built on a high-security architecture, AdVard avoids third-party agent installations on the smartphone by leveraging Android's native **Android Debug Bridge (ADB)** protocol via binary subprocess execution.

Layer	Technology Stack	Responsibility & Functionality
Frontend UI	React 19, TypeScript, Tailwind CSS, Lucide React	Renders dark-mode UI, manages real-time component state, captures user interactions (mouse clicks/drags, dialpad input, terminal commands).
IPC Layer	Electron ContextBridge, IPC Main / Renderer	Securely exposes typed asynchronous methods to the frontend without exposing Node process or shell access.
Backend ADB Layer	Node.js (child_process.spawn)	Resolves ADB binary paths across macOS/Win/Linux, formats non-injected arguments, pipes binary streams (screencap).
Target Device	Android OS (adbd daemon)	Receives adb commands over USB and executes kernel/framework calls (input keyevents, settings, package manager).

2. Detailed Functionality & Working Mechanism

2.1 Automated USB Device Detection

Working Mechanism: Continuously polls `adb devices` every 2 seconds via a non-blocking background loop. Parses serial numbers and device states (`device`, `unauthorized`, `offline`). Automatically updates the sidebar list and selects active devices.

```
adb devices
```

2.2 Comprehensive Device Specifications

Working Mechanism: Queries system properties in parallel using `getprop` and system dump services to extract detailed device information without UI lag.

```
adb -s SERIAL shell getprop ro.product.model
adb -s SERIAL shell getprop ro.build.version.release
adb -s SERIAL shell dumpsys battery
adb -s SERIAL shell wm size
adb -s SERIAL shell wm density
```

2.3 Hardware Navigation & Power Control

Working Mechanism: Executes hardware Android keyevents directly into the Linux input subsystem via `input keyevent`.

```
adb -s SERIAL shell input keyevent 3 # Home Button
adb -s SERIAL shell input keyevent 4 # Back Button
adb -s SERIAL shell input keyevent 187 # Recent Apps Button
adb -s SERIAL shell input keyevent 26 # Power Toggle
adb -s SERIAL reboot # Soft Reboot
```

2.4 Remote Device Unlock

Working Mechanism: Sends a sequence of wake-up events, automated swipe gestures, and optional security PIN inputs followed by an ENTER keyevent.

```
adb -s SERIAL shell input keyevent 224 # Wake Screen
adb -s SERIAL shell input swipe 500 1500 500 500 300 # Swipe Up
adb -s SERIAL shell input text # Input PIN
adb -s SERIAL shell input keyevent 66 # Press Enter
```

2.5 Interactive Live Screen Mirroring & Laptop Remote Control

Working Mechanism: Establishes a high-frequency frame capture stream (`screencap -p`). Calculates click/drag coordinates on the scaled HTML image canvas and maps them to actual Android screen resolutions for instant tap and swipe execution.

```
adb -s SERIAL exec-out screencap -p # Binary PNG Stream
adb -s SERIAL shell input tap # Laptop Mouse Click -> Tap
adb -s SERIAL shell input swipe # Laptop Mouse Drag -> Swipe
```

2.6 Phone Call Management

Working Mechanism: Allows starting, answering, and ending voice calls directly from the desktop application using Android Intent actions and Call keyevents.

```
adb -s SERIAL shell am start -a android.intent.action.CALL -d tel: # Make Call
adb -s SERIAL shell input keyevent 5 # Answer Call (KEYCODE_CALL)
adb -s SERIAL shell input keyevent 6 # End Call (KEYCODE_ENDCALL)
```

2.7 Volume, Media Playback & Brightness Control

Working Mechanism: Controls device audio streams, media player state, and system display brightness settings.

```
adb -s SERIAL shell input keyevent 24 # Volume Up
adb -s SERIAL shell input keyevent 25 # Volume Down
adb -s SERIAL shell input keyevent 164 # Mute/Unmute
adb -s SERIAL shell input keyevent 85 # Media Play/Pause
adb -s SERIAL shell settings put system screen_brightness <0-255>
```

2.8 Application Listing & Remote Launcher

Working Mechanism: Features tabbed filtering between Third-Party Apps (`-3`), System Apps (`-s`), or All Apps. Supports 1-click launch, force stopping running processes, clearing app cache/data, installing desktop `.apk` files, and uninstalling packages.

```
adb -s SERIAL shell pm list packages -3 # List User Apps
adb -s SERIAL shell monkey -p -c ... 1 # Launch App
adb -s SERIAL shell am force-stop # Force Stop App
adb -s SERIAL shell pm clear # Clear App Data
adb -s SERIAL install -r # Install APK
```

2.9 Embedded Interactive ADB Shell Terminal

Working Mechanism: Provides a desktop developer terminal to run direct ADB shell commands. Maintains command history (navigable via ↑/↓ keys), separates stdout and stderr streams, displays exit code badges, and allows copying command output.

```
adb -s SERIAL shell
```

2.10 Screenshot Capture & Export

Working Mechanism: Pipes raw binary PNG screenshot data from the device directly into Electron memory using `exec-out screencap -p` and provides a zoomable preview with desktop save dialog export.

```
adb -s SERIAL exec-out screencap -p
```

3. Security Architecture & Error Resiliency

1. Process Isolation: The renderer process runs with contextIsolation: true and nodeIntegration: false. No direct access to child_process, fs, or exec is permitted from React.
2. Command Injection Prevention: All ADB calls are executed using Node's child_process.spawn with explicit string array arguments rather than shell string interpolation.
3. Fault Tolerant Diagnostics: The app detects ADB installation paths automatically, provides 1-click ADB server restarts (`adb kill-server && adb start-server`), and handles `unauthorized` and `offline` state transitions gracefully.